PROJECT GUIDE

spend&Track

AI-assisted personal expense tracker — React frontend · FastAPI backend · PostgreSQL

React 18 + React Router 6 | Tailwind CSS | Chart.js | FastAPI + Uvicorn | psycopg 3 (async pool) | PostgreSQL 16 | JWT + bcrypt | Google OAuth | Gemini 2.0 Flash | PWA

1. Overview & scope
2. System architecture (flow diagram)
3. Backend request lifecycle
4. Folder structure
5. Feature list & logic flows
6. Backend — file & function reference
7. Frontend — file & function reference
8. API reference (endpoint → controller → UI caller)
9. Database schema
10. Configuration & running
11. Observations & notes

Generated 1 October 2026 from the repository at `spendandtrack/`. The Node/Express backend (`backend/`) is out of scope; this guide covers `frontend/` and `backend-fastapi/`.

1. Overview & scope

spend&Track (internally "spendAI") lets a user log expenses in plain text such as `250 uber`, auto-categorises them (keyword rules first, Gemini AI as a fallback), shows monthly statistics, a category doughnut chart and a 10-day trend chart, tracks per-category budget limits with alerts, and answers natural-language questions about spending through Gemini. It can be used with an account (email/password or Google) or as a **guest** whose data never leaves the browser. It is installable as a PWA.

Layer	Location	Technology
Frontend (SPA)	`frontend/`	React 18 (Create React App), React Router 6, Tailwind CSS 3 + inline styles, Chart.js 4 via react-chartjs-2, Axios, @react-oauth/google, service worker
Backend (REST API)	`backend-fastapi/`	FastAPI, Uvicorn, psycopg 3 + psycopg-pool (async), PyJWT, bcrypt, google-auth, httpx, python-dotenv
Database	`backend-fastapi/database/schema.sql`, `app/config/migrate.py`	PostgreSQL 16 (3 tables: users, expenses, budgets)
External services	—	Google Identity (OAuth sign-in), Google Gemini 2.0 Flash (called directly from the browser)
Infra	`backend-fastapi/Dockerfile`, `backend-fastapi/docker-compose.yml`, `frontend/nginx.conf`	Docker (db + FastAPI + React dev server), nginx config for a static production build

2. System architecture

Frontend Backend Database External

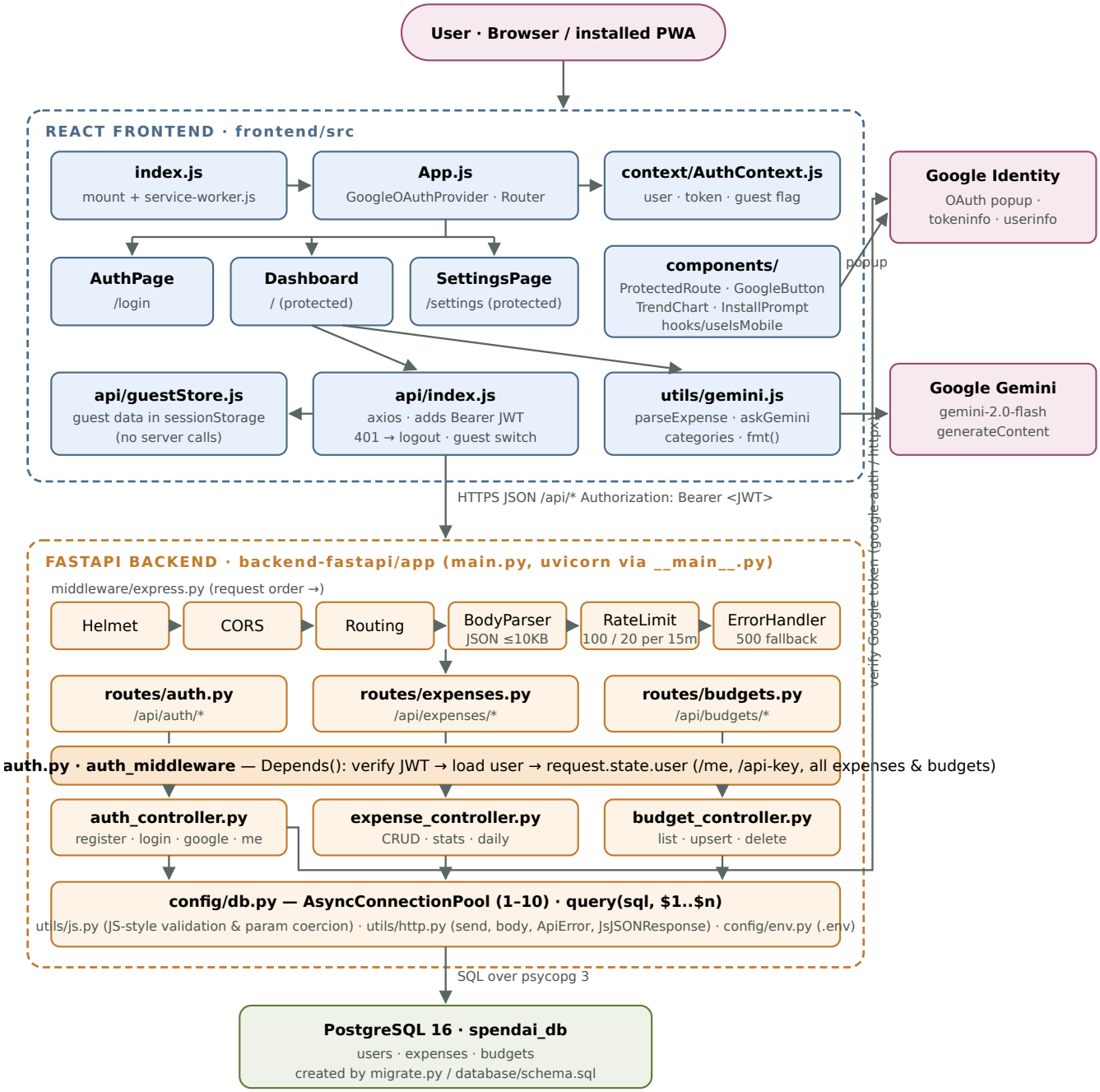


Figure 1 — End-to-end architecture. Guest sessions stop at `api/guestStore.js`; Gemini is called directly from the browser.

3. Backend request lifecycle

Every HTTP request to the FastAPI app passes through the same pipeline. Middleware is registered in `app/main.py` (Starlette runs the last-added first, so they are added innermost → outermost).

Browser — `api/index.js` request interceptor attaches `Authorization: Bearer <spendai_token>`
`frontend/src/api/index.js · api.interceptors.request`

▼

HelmetMiddleware — adds security headers (CSP, HSTS, X-Frame-Options, nosniff …)
`app/middleware/express.py`

▼

CORSMiddleware — allows `CLIENT_URL` with credentials; answers every `OPTIONS` preflight with 204 directly
`app/middleware/express.py`

▼

ExpressRoutingMiddleware — strips a trailing slash; serves `HEAD` via the `GET` route with an empty body
`app/middleware/express.py`

▼

BodyParserMiddleware — parses `application/json` into `request.state.body`; >10 KB or invalid JSON → 500 `{"error":"Internal server error"}`
`app/middleware/express.py · utils/http.py body()`

▼

RateLimitMiddleware — in-memory per-IP: 100 req / 15 min on `/api`, 20 / 15 min on `/api/auth`; over limit → 429 + `Retry-After`
`app/middleware/express.py · _Limiter.hit()`

▼

ErrorHandlerMiddleware — catches any unhandled exception → 500 JSON
`app/middleware/express.py`

▼

Router match (`routes/*.py`) → **Depends(auth_middleware)** for protected routes: decode JWT (HS256/384/512), load user row, set `request.state.user`; failures raise `ApiError(401, …)`
`app/routes/*.py · app/middleware/auth.py`

▼

Controller — validates input with `utils/js.py`, runs SQL with `db.query()`, returns `send(status, json)`
`app/controllers/*_controller.py`

▼

PostgreSQL — results come back as dicts; NUMERIC / BIGINT / UUID as strings, timestamps as ISO-8601 UTC strings
`app/config/db.py · _register_loaders, _JsDateLoader`

▼

Response — `JsJSONResponse` serialises like `JSON.stringify`; `ApiError` → `{"error": msg}`; unknown route → 404 `{"error":"Route not found"}`
`app/utils/http.py · app/main.py api_error_handler / not_found_handler`

> Why so much "Express" logic? `backend-fastapi` is a drop-in port of the original Express API: same URLs, JSON shapes, status codes, JWTs and bcrypt hashes. That is why the React frontend works against it with zero changes, and why helpers such as `utils/js.py` reproduce JavaScript semantics (`truthy`, `parseFloat`, `isNaN`).

4. Folder structure

```
spendandtrack/
├── frontend/                    React SPA
│   ├── public/                  index.html, manifest.json, service-worker.js, icons
│   ├── src/
│   │   ├── index.js             React root + service-worker registration
│   │   ├── App.js               Providers + routes
│   │   ├── index.css            Tailwind layers + fonts
│   │   ├── tailwindTheme.js     Colour/font tokens (Tailwind + inline styles)
│   │   ├── api/index.js         Axios client, auth/expenses/budgets APIs
│   │   ├── api/guestStore.js    Guest-mode in-browser backend
│   │   ├── context/AuthContext.js Auth state + actions
│   │   ├── components/          ProtectedRoute, GoogleButton, TrendChart, InstallPrompt
│   │   ├── hooks/useIsMobile.js Responsive breakpoint hook
│   │   ├── pages/               AuthPage, Dashboard, SettingsPage
│   │   └── utils/gemini.js      Expense parsing, AI Q&A, categories, currency format
│   ├── tailwind.config.js · postcss.config.js · nginx.conf · .env(.example)
│   └── package.json
├── backend-fastapi/             FastAPI REST API
│   ├── app/
│   │   ├── __main__.py          `python -m app` → uvicorn
│   │   ├── main.py              FastAPI app, middleware, error handlers, routers, /health
│   │   ├── config/              env.py, db.py (pool + query), migrate.py (schema)
│   │   ├── middleware/          auth.py (JWT guard), express.py (helmet/cors/body/rate-limit/errors)
│   │   ├── routes/              auth.py, expenses.py, budgets.py
│   │   ├── controllers/         auth_controller.py, expense_controller.py, budget_controller.py
│   │   └── utils/               http.py (responses), js.py (JS-compatible coercion)
│   ├── database/schema.sql      Standalone DB script (this release)
│   ├── requirements.txt · Dockerfile · docker-compose.yml · .env(.example)
├── docs/                        This guide (HTML source + PDF)
├── bin/                         Docker helper scripts (start/stop/wipe)
└── backend/                     Legacy Node/Express backend — ignored
```

5. Feature list & logic flows

#	Feature	Frontend	Backend / storage
F1	Email / password register & login	AuthPage.handleSubmit → AuthContext.register / login	POST /api/auth/register, /login → auth_controller.register / login
F2	Google sign-in	GoogleButton (useGoogleLogin popup) → AuthPage.handleGoogle → AuthContext.googleTokenLogin	POST /api/auth/google → google_auth → _get_google_profile
F3	Session restore & route protection	AuthProvider useEffect → getMe; ProtectedRoute; axios 401 interceptor	GET /api/auth/me → auth_middleware → get_me
F4	Guest mode (no account)	AuthPage.handleGuest → startGuest; api/guestStore.js; withGuestFallback	None — sessionStorage only, seeded with demo data
F5	Natural-language expense entry	Dashboard.addExpense → parseLocal → (parseExpense via Gemini if "Other")	POST /api/expenses → create_expense
F6	Expense list & delete	Dashboard.fetchAll, deleteExpense ("Recent Expenses")	GET /api/expenses, DELETE /api/expenses/{id}
F7	Month filter	Dashboard monthFilter select; months list from stats	?month=YYYY-MM on list & stats (TO_CHAR(date,'YYYY-MM'))
F8	Stat cards & category breakdown	StatCard, Doughnut chart, "Category Stats"	GET /api/expenses/stats/summary → get_stats
F9	10-day trend chart	components/TrendChart.js (prev/next, date picker, "Latest")	GET /api/expenses/stats/daily?from&to → get_daily
F10	Budget limits & alerts	Dashboard.addBudget, saveBudget, budgetAlerts	GET/POST /api/budgets → get_budgets / upsert_budget
F11	Ask AI about spending	Dashboard.doAsk → utils/gemini.askGemini	Direct call to Gemini (no backend)
F12	Settings & logout	SettingsPage (profile card, Sign out / Exit guest)	— (clears localStorage / sessionStorage)
F13	Installable PWA	index.js SW registration, public/service-worker.js, InstallPrompt	—
F14	Responsive / mobile layout	hooks/useIsMobile (640px / 900px), Tailwind breakpoints	—
F15	API security	—	JWT, bcrypt cost 12, helmet headers, CORS, rate limits, 10 KB body limit
F16	Health check	—	GET /health → {"status":"ok","timestamp"}

F1 · Email / password register & login

AuthPage form (mode = login | register) → `handleSubmit`
`pages/AuthPage.js`

▼

`AuthContext.register(name,email,password)` or `login(email,password)` → `authAPI.register/login`
`context/AuthContext.js · api/index.js`

▼

register: require name/email/password, password ≥ 6 chars → email already exists? 409 → bcrypt hash (cost 12, $2a$) → INSERT users → 201 {token, user}
`auth_controller.register · _hash_password · generate_token`

login: SELECT by lower(email) → not found 401 → no password_hash ("uses Google sign-in") 401 → bcrypt check → 200 {token, user without password_hash}
`auth_controller.login · _check_password · generate_token`

▼

`saveAuth()` clears guest data, stores `spendai_token` + `spendai_user` in localStorage, sets user → `navigate('/')`
`context/AuthContext.js`

JWT payload is `{userId, iat, exp}`, HS256 with `JWT_SECRET`; expiry from `JWT_EXPIRES_IN` (default `7d`, parsed by `_timespan_ms`).

F2 · Google sign-in

GoogleButton click → `useGoogleLogin()` opens Google OAuth popup (implicit flow) → returns `access_token`
`components/GoogleButton.js`

▼

`AuthPage.handleGoogle` → `googleTokenLogin(accessToken)` → POST `/api/auth/google {access_token}`
`pages/AuthPage.js · context/AuthContext.js`

▼

_get_google_profile: ID token (`credential`) → `id_token.verify_oauth2_token` ; access token → `tokeninfo` (aud must equal `GOOGLE_CLIENT_ID`) → `userinfo` (sub must match)
`auth_controller._get_google_profile`

▼

Email must be verified → find user by `google_id` → else by email (link: set google_id + avatar) → else INSERT new user → 200 {token, user}
`auth_controller.google_auth`

F3 · Session restore & route protection

App load: `AuthProvider` reads cached user; if a token exists calls `getMe()` (sets `loading` while waiting)
`context/AuthContext.js useEffect`

▼

GET /api/auth/me → `auth_middleware` (no token / expired / invalid / user deleted → 401) → `get_me` returns `request.state.user`
`middleware/auth.py · auth_controller.get_me`

▼

`ProtectedRoute` : loading → "Loading...", no user → `<Navigate to="/login">` . Any later 401 (except login/register/google) → interceptor clears storage and redirects to /login
`components/ProtectedRoute.js · api/index.js response interceptor`

F4 · Guest mode

"Just looking? Try guest mode" → `handleGuest` → `startGuest()` : removes JWT, sets `spendai_guest=1` in sessionStorage, seeds 21 demo expenses + 4 budgets
`pages/AuthPage.js · context/AuthContext.js · api/guestStore.js startGuestSession/buildSeed`

▼

`withGuestFallback()` routes every `expensesAPI` / `budgetsAPI` call to `guestExpensesAPI` / `guestBudgetsAPI` , which mirror the server's validation and response shapes (wrapped as `{data}` like axios)
`api/index.js · api/guestStore.js`

▼

Logout / closing the tab wipes the data (`clearGuestSession`). The server is never contacted.

F5 · Natural-language expense entry

User types `"250 chai"` → Enter / Add → `Dashboard.addExpense()`
`pages/Dashboard.js`

▼

parseLocal(text): regex `^(\d+(\.\d+)?)\s+(.+)$` → amount + description; keyword regexes → category (Food, Transport, Shopping, Health, Entertainment, Recharge, Education, Grocerries) else "Other"; emoji from `CAT_ICONS` ; date = today. Invalid → alert
`pages/Dashboard.js parseLocal`

▼

Only if category is **"Other"** and `REACT_APP_GEMINI_API_KEY` is set: `parseExpense()` asks Gemini for `{amount, category, description, emoji}` ; on failure falls back to its own keyword categoriser
`utils/gemini.js parseExpense · localCategorize`

▼

POST /api/expenses → `create_expense` : amount & description required, amount > 0 → INSERT (defaults: category "Other", emoji 📦, date NOW()) → 201 {expense}
`controllers/expense_controller.py`

▼

Clear input → `fetchAll()` refreshes list, stats, budgets and bumps `dataVersion` so TrendChart reloads

F6–F8 · Dashboard data load, month filter, stats

`fetchAll()` runs on mount and whenever `monthFilter` changes (default = current month) — three calls in parallel
`pages/Dashboard.js`

▼

GET /api/expenses?month → `get_expenses` : dynamic WHERE (month, category), ORDER BY date DESC, LIMIT 100 OFFSET 0

GET /api/expenses/stats/summary?month → `get_stats` : 5 queries via `asyncio.gather` — totals, byCategory, last-14-days daily, thisMonth, distinct months

GET /api/budgets → `get_budgets` ordered by category

▼

Derived on the client: top category, avg/day = total ÷ activeDays, doughnut data with `CAT_COLORS` , budget map, budget alerts. Delete button → `deleteExpense(id)` → DELETE /api/expenses/{id} (scoped to user; 404 if not owned)

F9 · 10-day trend chart

`windowBounds(monthFilter, months)` limits the 10-day window to the selected month and never past today; default window ends today
`components/TrendChart.js`

▼

‹ / › shift by 10 days, native date picker (`showPicker()`) chooses a start, "Latest" resets. Each change → `expensesAPI.getDaily({from,to})`

▼

GET /api/expenses/stats/daily → `get_daily` : validates YYYY-MM-DD and from ≤ to → SUM(amount) GROUP BY day in [from, to]

▼

Missing days filled with 0, today's bar highlighted, period total shown

F10 · Budget limits & alerts

"Add budget" (category select + amount) → `addBudget()` ; per-card edit → `saveBudget(cat)` ; both call `budgetsAPI.upsert`
`pages/Dashboard.js`

▼

POST /api/budgets → `upsert_budget` : valid category + amount > 0 → `INSERT … ON CONFLICT (user_id, category) DO UPDATE`
`controllers/budget_controller.py`

▼

`budgetAlerts` : spent ÷ budget ≥ 100% → red "exceeded"; ≥ 80% → amber "x% — ₹y left". Progress bars per budget card

F11 · Ask AI

Question → `doAsk()` (disabled when no `REACT_APP_GEMINI_API_KEY`)
`pages/Dashboard.js`

▼

`askGemini(question, stats, budgets, key)` builds a summary (total, count, per-category totals, budgets) and prompts Gemini for a 2–3 sentence answer in ₹
`utils/gemini.js`

F13 · Installable PWA

Production build only: `index.js` registers `/service-worker.js` . SW caches the app shell, uses network-first for navigations and cache-first for `/static/*` ; API (other origin) is never cached
`src/index.js · public/service-worker.js`

▼

`InstallPrompt` : Chrome/Android → captures `beforeinstallprompt` and shows "Install"; iOS Safari → shows Share → Add to Home Screen instructions; dismissal remembered 7 days
`components/InstallPrompt.js`

6. Backend — file & function reference

All paths relative to `backend-fastapi/app/`.

__main__.py — server entry point

Function	What it does	Used by
`main()`	Starts uvicorn with `app.main:app` on `HOST`/`PORT` (default 0.0.0.0:5000); `--reload` for dev	`python -m app`, Dockerfile CMD, docker-compose command

main.py — application assembly

Symbol	What it does	Used by
`lifespan()`	Opens the DB pool on startup, closes it on shutdown	FastAPI app startup/shutdown
`app`	FastAPI instance; JsJSONResponse default; docs/openapi disabled; no slash redirects	uvicorn
middleware registration	ErrorHandler, RateLimit (/api 100, /api/auth 20 per 15 min), BodyParser, ExpressRouting, CORS(`CLIENT_URL`), Helmet	Every request
`api_error_handler()`	Turns `ApiError` into `{"error": message}` with its status	Raised by `auth_middleware`
`not_found_handler()`	404/405 → `{"error":"Route not found"}`; for unknown paths under /api/expenses or /api/budgets it runs auth first so they 401 without a token	Starlette HTTPException
`health()`	GET /health → status + ISO timestamp	Monitoring / docker checks
`include_router()` ×3	Mounts auth, expenses, budgets routers	—

config/env.py

Function	What it does	Used by
`load_env()`	Loads `./.env` without overriding real environment variables (runs on import)	Imported first by main.py, db.py, __main__.py

config/db.py — database access

Symbol	What it does	Used by
`_TS_RE`, `_iso()`, `_pg_datetime_to_iso()`	Convert Postgres timestamp/date text into ISO-8601 UTC strings (…Z)	`_JsDateLoader`
`_JsDateLoader`	psycopg loader applying the conversion above	`_register_loaders`
`_register_loaders()`	Returns numeric/int8/uuid as strings and timestamps as ISO strings (what the frontend expects)	`_configure`, `connect`
`_connect_kwargs()`	autocommit, no prepared statements, `sslmode=require` when `NODE_ENV=production`	`pool`, `connect`
`_configure()`	Per-connection setup hook for the pool	`pool`
`pool`	`AsyncConnectionPool` min 1 / max 10, 10 s timeout, opened in lifespan	`query()`, main.lifespan
`_to_psycopg(sql, params)`	Rewrites `$1..$n` placeholders to psycopg named params; coerces values via `js.to_pg`	`query()`
`query(sql, params)`	Runs SQL on a pooled connection, returns object with `.rows` (list of dicts)	All controllers, `auth_middleware`
`connect()`	Standalone connection	`migrate.py`

config/migrate.py

Function	What it does	Used by
`migrate()`	CREATE TABLE IF NOT EXISTS users / expenses / budgets + 3 indexes; exits 1 on failure	`python -m app.config.migrate` (run on every docker-compose start). Same DDL as `database/schema.sql`

middleware/auth.py

Function	What it does	Used by
`auth_middleware(request)`	Requires `Bearer` header → `jwt.decode` (HS256/384/512) → SELECT user (id, name, email, avatar_url, gemini_api_key) → `request.state.user`. Errors: "No token provided", "Token expired", "Invalid token", "User not found" (all 401)	Depends on /api/auth/me, /api/auth/api-key, entire expenses & budgets routers; main.not_found_handler

middleware/express.py — HTTP pipeline

Symbol	What it does	Used by
`_respond()`, `_with_headers()`	Send a response from middleware / wrap send to edit headers	All middleware classes below
`_mounted(path, prefix)`	Case-insensitive prefix match on segment boundary	RateLimitMiddleware, main.not_found_handler

`HelmetMiddleware`	Adds `HELMET_HEADERS` (CSP, HSTS, COOP, CORP, nosniff, X-Frame-Options…)	main.py
`_vary()`, `CORSMiddleware`	CORS for `CLIENT_URL` with credentials; full preflight handling	main.py
`ExpressRoutingMiddleware`	Optional trailing slash; HEAD → GET without body	main.py
`BodyParserMiddleware` / `_parse()`	Reads JSON body (≤ 10 KB, must be object/array, rejects NaN/Infinity) into `scope.state.body`; replays body downstream	main.py; read by `utils/http.body()`
`_Limiter.hit()`, `RateLimitMiddleware`	Fixed-window per-IP counters; sets `X-RateLimit-*` headers; 429 when exceeded	main.py
`ErrorHandlerMiddleware`	Logs unhandled exceptions, returns 500 if nothing was sent yet	main.py

routes/*.py — URL → controller wiring

File	Routes	Notes
`routes/auth.py`	POST /register, /login, /google · GET /me · PATCH /api-key (prefix /api/auth)	/me and /api-key use Depends(auth_middleware)
`routes/expenses.py`	GET, POST "" · PUT, DELETE /{id} · GET /stats/summary · GET /stats/daily (prefix /api/expenses)	Router-level auth dependency; `PREFIX` used by main.py
`routes/budgets.py`	GET, POST "" · DELETE /{category:path}	`_delete_budget()` decodes the raw path so `Food%2FDrinks` works; literal "/" → 404

controllers/auth_controller.py

Function	What it does	Used by
`_timespan_ms(value)`	Parses "7d", "12h", "30m" … into milliseconds	`generate_token`
`generate_token(user_id)`	HS256 JWT `{userId, iat, exp}` signed with JWT_SECRET	register, login, google_auth
`_password_bytes()`	UTF-8 encode, truncate to 72 bytes (bcrypt limit)	_hash_password, _check_password
`_hash_password()`	bcrypt cost 12, $2a$ prefix, in threadpool	register
`_check_password()`	bcrypt verify in threadpool	login
`_log_error()`	Prints label + traceback	register, login, google_auth
`register()`	POST /api/auth/register — see F1	AuthContext.register
`login()`	POST /api/auth/login — see F1	AuthContext.login
`_get_google_profile()`	Verifies Google ID token or access token, returns sub/email/verified/name/picture	google_auth
`google_auth()`	POST /api/auth/google — find / link / create user — see F2	AuthContext.googleTokenLogin (and googleLogin)
`get_me()`	GET /api/auth/me → current user	AuthProvider on app load
`update_api_key()`	PATCH /api/auth/api-key → saves/clears `users.gemini_api_key`	AuthContext.updateApiKey (no page calls it currently)

controllers/expense_controller.py

Function	What it does	Used by (frontend)
`get_expenses()`	GET /api/expenses — filters `month`, `category`; `limit` (100) / `offset` (0); newest first	Dashboard.fetchAll
`create_expense()`	POST /api/expenses — validation + INSERT with defaults	Dashboard.addExpense
`update_expense(id)`	PUT /api/expenses/{id} — partial update via COALESCE; 404 if not the user's	expensesAPI.update (no UI yet)
`delete_expense(id)`	DELETE /api/expenses/{id}; 404 if not the user's	Dashboard.deleteExpense
`get_stats()`	GET /api/expenses/stats/summary — total, count, byCategory, daily (14 d), activeDays, thisMonth, months	Dashboard.fetchAll
`get_daily()`	GET /api/expenses/stats/daily?from&to — per-day totals in range	TrendChart

controllers/budget_controller.py

Function	What it does	Used by (frontend)
`get_budgets()`	GET /api/budgets ordered by category	Dashboard.fetchAll
`upsert_budget()`	POST /api/budgets — insert or update on (user_id, category)	Dashboard.addBudget, saveBudget
`delete_budget(category)`	DELETE /api/budgets/{category}	budgetsAPI.delete (no UI yet)

utils/http.py & utils/js.py

Symbol	What it does	Used by
`_js_jsonable()`, `JsJSONResponse`	JSON like `JSON.stringify` (NaN → null, 5.0 → 5), `charset=utf-8`	Default response class in main.py
`send(status, content)`	Shorthand for a JSON response	Every controller, middleware, handlers
`ApiError`	Exception carrying status + message	auth_middleware → main.api_error_handler
`body(request)`	Parsed JSON body as a dict (non-object → {})	Controllers

`truthy, trim, lower, length`	JS truthiness and string helpers	Controllers' validation
`to_number, is_nan, parse_float, parse_int`	JS `Number()`, `isNaN`, `parseFloat`, `parseInt`	expense/budget validation, stats
`to_string, number_to_string`	JS `String()` conversion	auth_controller, to_pg
`to_pg, _date_to_pg, _array_to_pg`	Serialise Python values as untyped Postgres text params	db._to_psycopg

7. Frontend — file & function reference

All paths relative to `frontend/src/` unless noted.

Entry & routing

File / symbol	What it does	Used by
`index.js`	Renders <App/> in StrictMode; registers service worker in production	`public/index.html` #root
`App()` (App.js)	GoogleOAuthProvider(`REACT_APP_GOOGLE_CLIENT_ID`) → AuthProvider → BrowserRouter. Routes: `/login` AuthPage, `/` Dashboard*, `/settings` SettingsPage*, * → /. (*ProtectedRoute). Renders InstallPrompt globally	index.js
`index.css`, `tailwindTheme.js`, `tailwind.config.js`	Design tokens (dark green palette, Syne / DM Mono fonts) shared by Tailwind classes and inline styles (`appTheme` as T)	All pages/components

api/index.js — HTTP client

Symbol	What it does	Used by
`api` (axios)	baseURL `${REACT_APP_API_URL}/api`, 15 s timeout	All API objects below
request interceptor	Adds `Authorization: Bearer` from localStorage `spendai_token`	Every call
response interceptor	401 (not on login/register/google) → clear storage, redirect /login	Every call
`authAPI`	register, login, googleAuth, googleAuthToken, getMe, updateApiKey	AuthContext
`serverExpensesAPI`	getAll, create, update, delete, getStats, getDaily	via `expensesAPI`
`serverBudgetsAPI`	getAll, upsert, delete (URL-encodes category)	via `budgetsAPI`
`withGuestFallback()`	Per-call switch: guest session → guestStore, else server	builds `expensesAPI`, `budgetsAPI`
`expensesAPI`, `budgetsAPI`	Public APIs used by the UI	Dashboard, TrendChart

api/guestStore.js — guest backend

Symbol	What it does	Used by
`GUEST_USER`, `isGuestSession()`	Fake user object; checks `spendai_guest` flag in sessionStorage	AuthContext, api/index.js
`store`, `load()`, `save()`	Safe sessionStorage wrapper; load data (re-seeding if missing)	guest APIs
`buildSeed()`, `SEED_EXPENSES`, `SEED_BUDGETS`	21 demo expenses spread over 45 days, 4 budgets	startGuestSession, load
`startGuestSession()`, `clearGuestSession()`	Begin / wipe guest mode	AuthContext.startGuest, saveAuth, logout
`newId, pad, localMonth, localDay, daysAgo, toTimestamp, byDateDesc, inMonth, ok, fail`	IDs, local-time date keys, sorting/filtering, axios-like result helpers	Internal
`guestExpensesAPI`	getAll, create, update, delete, getStats, getDaily — same rules & shapes as backend	withGuestFallback
`guestBudgetsAPI`	getAll, upsert, delete	withGuestFallback

context/AuthContext.js — auth state

Symbol	What it does	Used by
`AuthProvider`	Holds `user`, `loading`, `isGuest`; validates stored token with `getMe()` on mount	App.js
`saveAuth(token, user)`	Clears guest data, persists token + user	register, login, googleLogin, googleTokenLogin
`register / login`	Call authAPI and save auth	AuthPage.handleSubmit
`googleTokenLogin`	Exchange Google access token for app JWT	AuthPage.handleGoogle
`googleLogin`	Same with a Google ID token (credential)	Not used by current UI
`startGuest`	Removes JWT, starts guest session	AuthPage.handleGuest
`logout`	Clears guest + JWT storage, user = null	Dashboard top bar, SettingsPage
`updateApiKey`	PATCH user's Gemini key (skipped for guests)	Not used by current UI
`useAuth()`	Context hook; throws outside provider	Pages, ProtectedRoute

pages/

Symbol	What it does	Used by
`AuthPage`	Login / register toggle form, password show/hide, Google button, guest button, error display	Route /login
AuthPage · `Logo`, `DashboardPreview`, `Field`	Brand mark; decorative mini-dashboard on the brand panel (≥ lg screens); labelled input wrapper	AuthPage

AuthPage · `switchMode, update, handleSubmit, handleGoogle, handleGuest`	Form state + the three sign-in paths (F1, F2, F4)	AuthPage JSX
`Dashboard`	Main screen: top bar (month select, settings, logout), Log Expense, stat cards, Category Breakdown doughnut, TrendChart, Recent Expenses (30), Category Stats, Budget Limits, Ask AI	Route /
Dashboard · `StatCard`, `currentMonthKey`	KPI tile; "YYYY-MM" of today	Dashboard
Dashboard · `fetchAll`	Parallel load of expenses, stats, budgets (F6–F8)	useEffect, after every mutation
Dashboard · `parseLocal`, `addExpense`	Text → expense, with Gemini fallback (F5)	Log Expense input / button
Dashboard · `deleteExpense`	Delete + refresh	× button on each expense
Dashboard · `saveBudget`, `addBudget`, `budgetAlerts`, `budgetCards`	Budget editing and alert computation (F10)	Budget Limits panel, alerts banner
Dashboard · `doAsk`	Ask Gemini (F11)	Ask AI panel
`SettingsPage`	Account card (avatar/initial, name, email), Sign out / Exit guest mode	Route /settings

components/, hooks/, utils/

Symbol	What it does	Used by
`ProtectedRoute`	Loading screen → children if logged in (or guest) → else redirect /login	App.js routes / and /settings
`GoogleButton` (+ `GoogleMark`, `Spinner`)	Custom-styled Google popup sign-in; handles popup blocked / cancel	AuthPage
`TrendChart`	10-day bar chart with navigation (F9)	Dashboard
TrendChart · `toKey, fromKey, addDays, clamp, monthEnd, shortDate, windowBounds, btn`	Local-date arithmetic, window limits, button styles	TrendChart
`InstallPrompt` (+ `isStandalone, isIOS, isIOSSafari, recentlyDismissed, ShareIcon`)	PWA install banner (F13)	App.js
`useIsMobile(maxWidth=640)`	Reactive `matchMedia` breakpoint	Dashboard (640 & 900), TrendChart
`parseExpense(text, key)`	Gemini expense parser with keyword fallback	Dashboard.addExpense
`localCategorize(desc)`, `CAT_KW`	Keyword → category	parseExpense
`askGemini(q, stats, budgets, key)`	Spending Q&A prompt to Gemini	Dashboard.doAsk
`CAT_ICONS`, `CAT_COLORS`	Category emoji and chart colours	Dashboard, parseLocal
`fmt(n)`	₹ with Indian digit grouping, rounded	Dashboard, TrendChart

public/ & config

File	What it does	Used by
`public/service-worker.js`	install (cache shell), activate (drop old caches), fetch (network-first pages, cache-first /static)	index.js (production)
`public/manifest.json`, icons	PWA name, colours, icons (192/512/maskable)	Browser install
`nginx.conf`	SPA fallback to index.html; no-cache for html/json, 1-year cache for assets	Static production hosting
`.env`	`REACT_APP_API_URL`, `REACT_APP_GOOGLE_CLIENT_ID`, `REACT_APP_GEMINI_API_KEY`	Build time (CRA)

8. API reference

Method	Path	Auth	Controller	Request → Response	Frontend caller
GET	/health	—	main.health	→ {status, timestamp}	—
POST	/api/auth/register	—	register	{name,email,password} → 201 {token,user}	AuthContext.register
POST	/api/auth/login	—	login	{email,password} → {token,user}	AuthContext.login
POST	/api/auth/google	—	google_auth	{credential} or {access_token} → {token,user}	AuthContext.googleTokenLogin
GET	/api/auth/me	JWT	get_me	→ {user}	AuthProvider (mount)
PATCH	/api/auth/api-key	JWT	update_api_key	{gemini_api_key} → {message}	AuthContext.updateApiKey (unused)
GET	/api/expenses	JWT	get_expenses	?month&category&limit&offset → {expenses}	Dashboard.fetchAll
POST	/api/expenses	JWT	create_expense	{amount,description,category?,emoji?,date?} → 201 {expense}	Dashboard.addExpense
PUT	/api/expenses/{id}	JWT	update_expense	partial fields → {expense}	(not in UI)
DELETE	/api/expenses/{id}	JWT	delete_expense	→ {message}	Dashboard.deleteExpense
GET	/api/expenses/stats/summary	JWT	get_stats	?month → {total,count,byCategory,daily,activeDays,thisMonth,months}	Dashboard.fetchAll
GET	/api/expenses/stats/daily	JWT	get_daily	?from&to → {daily:[{day,total}]}	TrendChart
GET	/api/budgets	JWT	get_budgets	→ {budgets}	Dashboard.fetchAll
POST	/api/budgets	JWT	upsert_budget	{category,amount} → {budget}	Dashboard.addBudget / saveBudget
DELETE	/api/budgets/{category}	JWT	delete_budget	→ {message}	(not in UI)

Errors always have the shape `{"error": "message"}`. Common statuses: 400 validation, 401 auth, 404 not found, 409 email exists, 429 rate limited, 500 server error.

9. Database schema

Script: `backend-fastapi/database/schema.sql` (idempotent; same DDL as `app/config/migrate.py`). Run with `psql "$DATABASE_URL" -f backend-fastapi/database/schema.sql`.

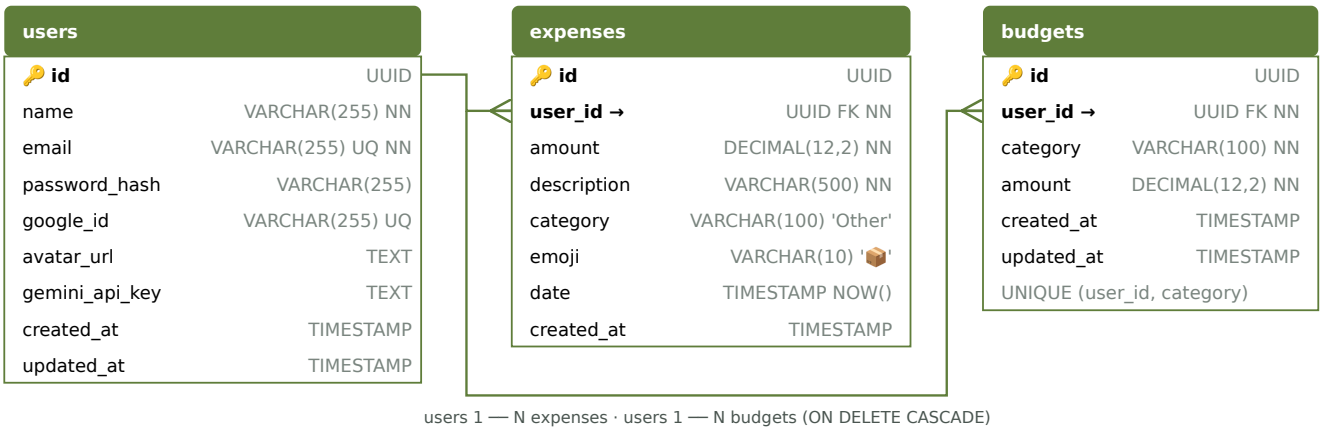


Figure 2 — Entity-relationship diagram. Indexes: idx_expenses_user_id, idx_expenses_date, idx_budgets_user_id.

Query (controller)	SQL pattern
Month filter	`TO_CHAR(date,'YYYY-MM') = $n`
Stats totals	`COUNT(*), COALESCE(SUM(amount),0), COUNT(DISTINCT date::date)`
By category	`GROUP BY category ORDER BY total DESC` (emoji = MIN(emoji))
Daily totals	`TO_CHAR(date,'YYYY-MM-DD') as day … GROUP BY day`
Budget upsert	`ON CONFLICT (user_id, category) DO UPDATE SET amount=$3, updated_at=NOW()`
Ownership	Every expense/budget query includes `user_id = request.state.user.id`

10. Configuration & running

Backend environment — `backend-fastapi/.env`

Variable	Purpose
`PORT` / `HOST`	Uvicorn bind (default 5000 / 0.0.0.0)
`NODE_ENV`	`production` turns on DB SSL (name kept for compatibility)
`DATABASE_URL`	postgresql://user:pass@host:5432/spendai_db
`JWT_SECRET`, `JWT_EXPIRES_IN`	JWT signing secret and lifetime (e.g. 7d)
`GOOGLE_CLIENT_ID`	Must match the frontend's client ID (token audience check)
`CLIENT_URL`	Allowed CORS origin (frontend URL)

Frontend environment — `frontend/.env`

Variable	Purpose
`REACT_APP_API_URL`	Backend base URL without `/api` (e.g. http://localhost:5000)
`REACT_APP_GOOGLE_CLIENT_ID`	Google OAuth web client ID
`REACT_APP_GEMINI_API_KEY`	Enables Gemini parsing fallback and Ask AI

Run locally

```
# 1. Database
createdb spendai_db
psql "$DATABASE_URL" -f backend-fastapi/database/schema.sql     # or: python -m app.config.migrate

# 2. Backend
cd backend-fastapi
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
cp .env.example .env            # fill in values
.venv/bin/python -m app --reload   # http://localhost:5000

# 3. Frontend
cd frontend
npm install
npm start                       # http://localhost:3000

# Or everything with Docker (db + FastAPI + React):
cd backend-fastapi && docker-compose up -d
```

11. Observations & notes

- **Gemini key is a frontend build variable.** `REACT_APP_GEMINI_API_KEY` is bundled into the public JavaScript, so anyone can read it. For production, move Gemini calls behind a FastAPI endpoint (e.g. `POST /api/ai/parse`, `/api/ai/ask`) and keep the key server-side.
- **Per-user key is unused.** `users.gemini_api_key` and `PATCH /api/auth/api-key` still exist, but no page calls `updateApiKey` any more (Settings no longer has the key field).
- **Endpoints without UI:** `PUT /api/expenses/{id}` (edit expense) and `DELETE /api/budgets/{category}` (remove budget) are implemented in backend and API client but not exposed in the Dashboard.
- **Two categorisers.** `Dashboard.parseLocal` and `utils/gemini.localCategorize` have overlapping but different keyword lists; consolidating them into `utils/` would keep categories consistent.
- **Category spelling.** "Grocerries" is used as a category key in several places (CAT_ICONS, CAT_COLORS, seeds, parser); renaming requires a data migration for existing rows.
- **Root docker-compose.yml and `bin/*.sh`** still start the Node backend. Use `backend-fastapi/docker-compose.yml` for the React + FastAPI stack.
- **Rate limiting is in-memory** and per process; with multiple workers/instances each keeps its own counters.
- **Timezone.** `date` is `TIMESTAMP` (no zone); month/day grouping happens in the DB server's timezone while the guest store and TrendChart use the browser's local time.